

Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided $p = 1.0$; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.

I. INTRODUCTION

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate-validate-repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus

baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar $p = 1.0$. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. RELATED WORK

Recent surveys and prototypes explore LLMs for NetOps tasks including intent translation, configuration generation, and remediation planning; these works emphasize both potential productivity gains and safety concerns when LLMs propose executable changes [openalex:W7161354925; openalex:W7170714602; TechRxiv:177004277]. Prior empirical and architectural work shows that augmenting LLMs with deterministic tools (verifiers, simulators, guarded tool schemas) often improves measured correctness on curated task sets and can reduce constraint violations in produce-validate pipelines [openalex:W4412888330; openalex:W7161354925]. Red-teaming, guardrail, and MCP/schema proposals highlight trade-offs between missed unsafe proposals and increased false refusals, and they call for workflow-centred, replayable

evaluations with deterministic sandboxing and archived traces [TechRxiv:177006109; TechRxiv:177004277].

Our study is not a new verification algorithm or a multi-model leaderboard. Instead, it complements existing work by empirically interrogating how concrete experimental semantics—particularly generation budgeting and shared vs independent generation—interact with a paired estimand and can render a confirmatory design untestable. We therefore situate our contribution as a reproducible, methodological diagnosis that bridges architecture proposals and practical evaluation design.

III. METHODOLOGY

Preregistration and primary estimand. The experiment was preregistered as cand-2026-01-prereg-v1. The primary estimand is the paired binary difference `paired_success_rate_difference_guarded_minus_baseline` where each arm contributes a binary indicator per task (1 = SAFE, 0 = UNSAFE) derived from a deterministic sandbox validator. The confirmatory test is an exact two-sided McNemar implemented by the `paired_binary_analysis_v1` executor; paired bootstrap percentile confidence intervals use 10,000 resamples with seed = 1729. The preregistration specified a paired binary design with N=200 unique tasks (400 paired episodes across two arms under independent per-arm draws), power calculations targeting an ≈ 7 percentage-point paired reduction, and predeclared sensitivity and multiplicity procedures.

Task generation, scope, and deterministic seeding. A deterministic task generator (`deterministic_netops_task_generator_v5`) produced 200 synthetic intent→configuration tasks constrained to allowed fields {admin, mtu, vlan} within single-AS multi-device topologies. Each task manifest includes `initial_state`, `expected_final_state`, an explicit `required_sequence` (workflow policy), and difficulty metadata; generation was seeded deterministically (per-task `generation_seed` 1..200 as recorded in `execution/task_manifest.jsonl`) so every task can be reconstructed exactly. Tasks were classified into benign and adversarial clusters using a preregistered adversarial template set designed to increase difficulty by obfuscation, contradictory constraints, and privilege-escalation phrasing; the adversarial cluster formed the confirmatory challenge set alongside benign items per the preregistration.

Pipelines compared and validator semantics. Two pipelines were contrasted: (1) baseline (LLM-only): accept the generator output and evaluate post hoc with `deterministic_netops_validator_v5`; (2) guarded (LLM + deterministic validator): validate the generator output using `deterministic_netops_validator_v5` in a sandboxed replay and (optionally) invoke an automatic repair step once. For confirmatory episodes in this run the repair pathway was not exercised (`repair_provider_trace` fields are null in `execution/scoring/*.json`), so the guarded arm operationally reflects deterministic validation only. The validator applies the preregistered `scoring_rule_full_intent_constraint_validation` and emits deterministic traces

(`validation_before`, `validation_after`, violation lists, and a boolean valid flag) that produce the binary SAFE/UNSAFE outcome used by analysis. Representative validator traces are archived under `execution/scoring/*` and `execution/responses/*` and show the `validator_id` and `validator_version` recorded per episode.

Generation semantics, model budgeting, and execution accounting. The preregistration allowed two generation semantics: independent per-arm generation (two independent LLM calls per task, one for each arm) or `shared_initial_candidate` (one shared LLM call per task whose candidate is scored by both arms). To conserve hosted model budget the executed run used `shared_initial_candidate` semantics; this is recorded in the archived model configuration (`execution/model_configuration.json`: `independent_condition_generation` = false) and in the execution manifest (`execution/execution_manifest.json`: `model_calls_used` = 200). The preregistered independent per-arm design would have used up to 400 model calls (two per task) under the same N=200 confirmatory sample; that budget differential is therefore the precise additional model-call requirement to enable arm-level stochastic contrasts under the preregistered plan. The execution manifest and model configuration are archived and document these accounting facts.

Scoring, missingness, exclusions, and deterministic reconciliation. `Deterministic_netops_validator_v5` produced a deterministic valid flag per candidate; confirmatory binary labels were derived directly from `validator.valid`. The preregistration specified one automatic retry for model call failures, duplicate-prompt detection (prompt hashes lead to automatic exclusion+replacement), verifier nondeterminism checks (replays), and a stopping rule that aborts or downgrades if technical failures exceed 10%. In the archived confirmatory run, deterministic reconciliation artifacts (`analysis/deterministic_reconciliation.json` and `execution/execution_manifest.json`) report `completed_episode_count` = 400, `failed_episode_count` = 0, `complete_pairs` = 200, and that all deterministic consistency checks passed; `analysis/contamination_summary.csv` and `analysis/missingness_summary.csv` document zero exclusions or technical failures. Those artifacts confirm that the run executed under the shared generation semantics without technical interruptions and that no contaminated or nondeterministic tasks were present in the confirmatory set.

Analysis pipeline and confirmatory computation. The analysis executor `paired_binary_analysis_v1` consumed the per-episode binary outcomes and produced the 2×2 contingency (`analysis/paired_contingency_table.csv`: n11,n10,n01,n00), exact McNemar two-sided p-value, and paired bootstrap CI (`analysis/analysis_log.jsonl`). For this execution the contingency was n11 = 200, n10 = 0, n01 = 0, n00 = 0, yielding an estimated paired SAFE-rate difference = 0.0, exact McNemar p = 1.0, and paired bootstrap 95% CI = [0.0, 0.0]. Representative per-task

scoring JSONs and shared initial responses are archived under `execution/scoring/` and `execution/responses/` (e.g., `task-000001-*.json` and `task-000001-shared-initial.txt`) and show identical `shared_initial_candidate` content and validator traces with `violation_count = 0`. The preregistered exploratory reserve (up to 50 tasks to estimate verifier ROC/refusal trade-offs) was not consumed prior to confirmatory execution; consequently no ROC or refusal estimates appear in the confirmatory artifacts.

Design consequences and protocol fidelity. Because the executed semantics produced a single shared candidate per task that both arms scored deterministically and because the validator labeled every sampled candidate SAFE, the paired design produced no discordant pairs and therefore no testable paired effect under McNemar logic. This outcome is an execution-semantic and budgeting consequence that follows directly from archived configuration and execution artifacts (`execution/model_configuration.json` and `execution/execution_manifest.json`) rather than from properties of the validator algorithm. All files referenced above are included in the archived bundle so readers can replay generation, scoring, and analysis deterministically to confirm the chain of events described here.

IV. RESULTS

Execution accounting and provider audit. The execution manifest reports `completed_episode_count = 400` (200 paired episodes), `failed_episode_count = 0`, and `model_calls_used = 200`, consistent with the executed `shared_initial_candidate` semantics. Provider-level auditing recorded `hosted_model_call_event_count = 200` and `cache_miss_count = 200`; cross-condition cache reuse was not observed. No technical failures, exclusions, or nondeterministic verifier behaviors were recorded in the confirmatory set, and deterministic reconciliation checks passed, confirming that the archived episode and scoring artifacts correspond to a single, reproducible run.

Primary confirmatory outcomes and contingency. The deterministic validator produced SAFE labels for every confirmatory episode: baseline SAFE successes = 200/200 and guarded SAFE successes = 200/200. The resulting paired contingency counts are $n_{11} = 200$ (SAFE in both arms), $n_{10} = 0$ (SAFE guarded only), $n_{01} = 0$ (SAFE baseline only), and $n_{00} = 0$ (UNSAFE in both). From these counts the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test computed $p = 1.0$; the paired bootstrap percentile 95% confidence interval is degenerate at $[0.0, 0.0]$ (resamples = 10000; bootstrap_seed = 1729). These numerical values are direct outputs of `paired_binary_analysis_v1` and are archived in the analysis artifacts (`condition_summary` and `paired_contingency_table` files).

Representative diagnostic traces. Representative scoring JSONs and validator traces illustrate why the contingency is degenerate. For multiple sampled tasks the

`shared_initial_candidate` content (the raw generated configuration) is identical across baseline and guarded episodes; validator traces show `validation_before` and `validation_after` with `violation_count = 0` and `valid = true` in every case. The `repair_provider_trace` entries are null in all confirmatory scoring JSONs, confirming that no automatic repair was invoked and that the guarded arm performed only validation. Shared response files for representative tasks contain the canonical generated configurations used by both arms, reinforcing that no arm-specific generation stochasticity existed under the executed semantics.

Missingness, contamination, and exploratory reserves. `Analysis/missingness_summary` documents `complete_pairs = 200` with zero failed or unscorable episodes; `analysis/contamination_summary` shows no flagged episodes. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that exploratory reserve was not consumed prior to confirmatory execution, so no empirical ROC or refusal-rate estimates appear in the confirmatory analysis artifacts.

Statistical interpretation and inferential consequence. The observed ceiling outcome ($n_{11} = 200$ with no discordant pairs) renders the preregistered inferential machinery non-informative for the intended causal estimand: McNemar’s test relies on discordant pairs (n_{10} and n_{01}) to evaluate paired differences, and their absence produces a p -value of 1.0 and a degenerate CI at zero. This degeneracy does not demonstrate verifier effectiveness; rather, it indicates that under the executed shared generation semantics and for the sampled model outputs, there was no arm-level variation to attribute to the presence or absence of the deterministic validator. The preregistered power calculation had assumed a nonzero baseline unsafe rate ($\approx 10\text{--}15\%$) and discordant proportions; those assumptions were not met in the executed run because budgeted generation semantics constrained per-arm variability.

Archival fidelity and reproducibility pointers. All numerical summaries and the contingency table reported above are identical to the archived analysis artifacts (`analysis/condition_summary.csv` and `analysis/paired_contingency_table.csv`). Representative per-task scoring JSONs and shared response files show the identical generated candidate for baseline and guarded episodes and document validator decisions and violation lists. The execution and analysis logs capture the exact bootstrap parameters (resamples = 10000, seed = 1729) and the exact statistical outputs. Because the confirmatory run contains no technical failures or exclusions, the archived artifacts permit deterministic replay of the identical non-informative outcome; investigators seeking an informative repeat should consult these artifacts before changing preregistered design choices.

V. DISCUSSION

We expand the original diagnosis with technical detail and artifact-grounded interpretation to help others design informative repeats that avoid the ceiling/testability failure observed here.

Mechanism of the ceiling failure. The confirmatory estimand (paired_success_rate_difference_guarded_minus_baseline) depends on arm-level discordance: McNemar-style inference requires nonzero counts in the off-diagonal cells (n10 and/or n01). In our execution the pipeline semantics used shared_initial_candidate (one LLM output per task that both arms scored). Because no repairs were applied in confirmatory episodes and the validator gave identical deterministic verdicts for the shared candidate, every paired outcome fell into n11 (SAFE in both arms). This deterministic concordance eliminates the very data structure McNemar inference needs and yields a vacuous test ($p = 1.0$) and a degenerate bootstrap CI [0.0,0.0]. The archived artifacts that directly demonstrate this mechanism include execution/model_configuration.json (declared independent_condition_generation = false) and analysis/paired_contingency_table.csv (n11 = 200, n10 = n01 = n00 = 0).

Budgeting and statistical power implications. The preregistration explicitly considered two generation semantics and a target power calculation predicated on a nonzero baseline unsafe rate ($\approx 10\text{--}15\%$) and expected discordant proportions (e.g., preregistered $p_{10}=0.07$, $p_{01}=0.01$ leading to $N \approx 174$). Independent per-arm generation (two independent draws per task) is necessary to produce arm-level stochastic contrasts when the intervention (validator presence) is applied post-generation and repairs are not used to change candidates. Independent per-arm draws at our planned sample size would increase model calls from the executed 200 to the preregistered 400. The execution artifact model_calls_used = 200 versus the preregistered maximum_model_calls = 400 documents the exact budgetary shortfall that removed per-arm stochasticity and thus collapsed statistical sensitivity.

Role of repairs and alternative discordance mechanisms. Two possible, protocol-compatible mechanisms could have produced discordant pairs without independent per-arm draws: (1) enable guarded-arm repairs that modify a candidate flagged UNSAFE into a SAFE configuration and (2) use a verifier that can nondeterministically refuse while baseline accepts (or vice versa). In our archived confirmatory episodes the repair_provider_trace fields are null and no repairs were applied; consequently repairs were not a source of discordance. If repairs are allowed in the guarded arm, a different estimand (e.g., paired probability that guarded produces SAFE after repair while baseline remains UNSAFE) should be preregistered and analyzed with contingency cells defined appropriately. The scoring JSONs and repair fields in execution/scoring/*.json make the absence of repair activity explicit for this run.

Task difficulty and validator coverage diagnostics. Representative task manifests (archived in execution/task_manifest.jsonl) include per-task difficulty metadata. In the six representative tasks included in the manuscript bundle the preregistered difficulty labels are: medium (1), hard (2), very_hard (3). These mixed difficulty labels indicate the generator produced nontrivial task patterns (e.g., multi-step shutdown/configure/restore or make-before-break switchover)

even though the sampled gpt-5-mini outputs satisfied the validator on every confirmatory item. This suggests two nonexclusive interpretations consistent with archived evidence: (a) the LLM generated valid step sequences for these task templates reliably under the provided prompts and model setting; or (b) the deterministic_netops_validator_v5 abstractions and the synthetic task family together created a validation surface for which the model's current capabilities were sufficient. The validator traces (validation_before/after) in execution/scoring/*.json show required_command_count and parsed_command_count matching expected sequences for representative tasks, supporting the artifact-grounded observation that produced candidates met the formalized intent constraints for the sampled items. However, the validator's completeness relative to real-world NetOps failure modes remains a separate threat to external validity (see Limitations).

Practical design prescriptions supported by artifacts. From the recorded preregistration assumptions and execution manifest we make three concrete, archive-supported prescriptions for informative repeats: - Prefer independent per-arm generation when the estimand tests the effect of adding a deterministic verifier to generation without repairs. For $N=200$ paired tasks this requires roughly doubling model calls from 200 to ~ 400 (compare execution/model_configuration.json and execution/execution_manifest.json). This change restores per-arm stochasticity and permits nonzero off-diagonal counts to appear when the validator interacts with model variability. - Use the preregistered exploratory reserve (up to 50 tasks) to calibrate adversarial transforms until a nonzero baseline unsafe rate consistent with the power plan is observed. Archive the exploratory runs and ROC estimates before committing to the confirmatory run; the preregistration and analysis artifacts show that the exploratory reserve was available but unused here. - If repairs are permissible in the guarded arm, preregister a confirmatory estimand that explicitly accounts for repair conversions (UNSAFE $\rightarrow$ SAFE) and instrument repair_provider_trace fields for counting conversions and side effects; ensure repairs themselves are deterministically replayable and archived (repair traces were null in this execution—see execution/scoring/*.json). These prescriptions are not speculative: they are directly motivated by and supported in the archived preregistration, execution manifest, and analysis outputs.

Statistical interpretability and reporting guidance. When a ceiling or floor outcome appears (all observations in one cell), standard hypothesis tests produce noninformative outputs (exact $p = 1.0$, degenerate CIs). We recommend that authors (a) report the contingency table and raw counts prominently (analysis/paired_contingency_table.csv), (b) predeclare and run sensitivity analyses that treat technical failures as safe or unsafe per the preregistration, and (c) document any shared-generation semantics and model-call budget choices in the Methods (execution/model_configuration.json). Doing so makes it clear when a null statistical result owes to experimental semantics rather than substantive equivalence of

pipelines.

Operational implications and limits of inference. The present run is evidence that, for the sampled synthetic tasks and generation setting, gpt-5-mini produced validator-compliant configurations under the specified prompts; it is not evidence that deterministic verification is unnecessary in general. The degeneracy of the contingency table prevents statements about relative safety or missed unsafe proposals. Practitioners seeking to evaluate verifier utility in operational deployments should (i) decide whether the estimator of interest is effect of validator presence on independent generation or effect of repair conversion, (ii) allocate generation budget accordingly, and (iii) tune adversarial strength via held-out exploratory trials. All these recommendations are directly traceable to the archived preregistration and execution artifacts.

Reproducibility and artifact pointers. The diagnosis above is fully reproducible from the archived bundle: the generation semantics and model_call accounting appear in execution/model_configuration.json and execution/execution_manifest.json; the paired contingency and condition summaries are in analysis/paired_contingency_table.csv and analysis/condition_summary.csv; representative scoring traces are in execution/scoring/*.json. Researchers planning informative repeats should consult these artifacts first to avoid repeating the same semantic mismatch between estimand and execution.

VI. LIMITATIONS

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. CONCLUSION

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar $p = 1.0$; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

DISCLOSURE STATEMENT

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15. Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring

JSONs (execution/scoring/*.json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

- [1] <https://arxiv.org/abs/2605.12729>
- [2] <https://doi.org/10.2139/ssrn.7132138>
- [3] <https://doi.org/10.36227/techrxiv.177004277.71795055/v1>
- [4] <https://doi.org/10.36227/techrxiv.177006109.97957916/v1>
- [5] <https://doi.org/10.1109/ojcoms.2026.3680457>
- [6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
- [7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Shahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026